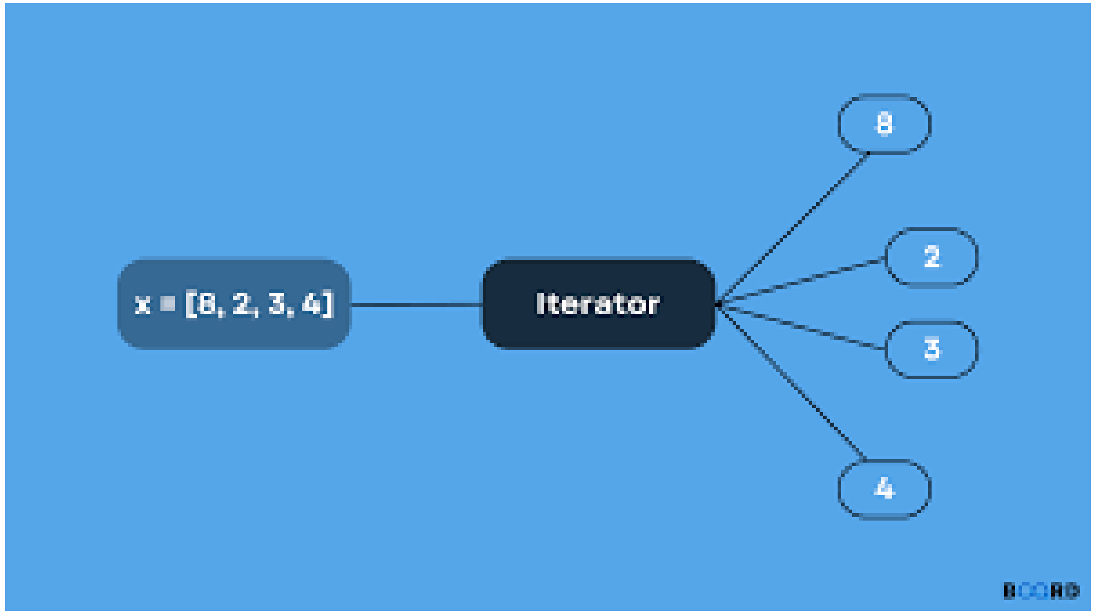
Phase 1, Module 10: Advanced Loops & Iteration

Khyber AI Systems Engineer (KASE)

Welcome to Module 10. Today, we go beyond the basic 'for' loop. We will learn how to handle parallel data streams, how to track indexes without manual counters, and how to write powerful one-line data transformations.

Presented By: Zaryab Ahmad

Iterables vs. Iterators



What's Under the Hood?

- **Iterable**: Any object you can loop over (List, String, Dictionary).
- **Iterator:** The actual object that tracks the "current position" and fetches the next item.
- **The next() Function:** How Python moves through a sequence.

Parallel & Indexed Iteration

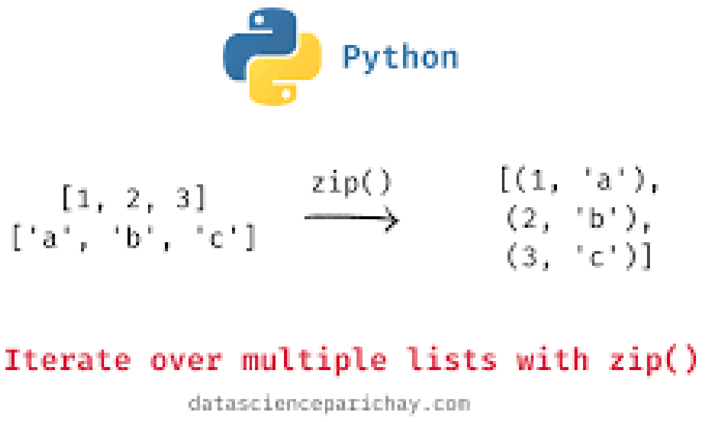
- ***enumerate*() and *zip()***

- ***enumerate*()**: Automatically tracks the index count while looping. No more i = 0 and i += 1.
- ***zip():*** Sews two or more lists together to process them in parallel.

```
lst = ('dog','cat','rat','donkey')

for x,y in enumerate(lst):
    print(x)
    print(y)
```

```
0
dog
1
cat
2
rat
3
donkey
```



List Comprehensions (Clean Code)

```python
names = ["Jim", "Audrey", "Clara", "Aaron", "Ishaan", "Amelia"]

long_names = [name for name in names if len(name) > 5]

print(long_names)
# OUTPUT: ['Audrey', 'Ishaan', 'Amelia']
```

The Power of the One-Liner

- A compact way to create new lists from old ones.
- **Syntax:** $[expression\ for\ item\ in\ iterable\ if\ condition]$
- **Performance:** They are executed at near-C speeds, making them faster than standard loops.